



UNIVERSITÉ DE BORDEAUX
COLLÈGE SCIENCES ET TECHNOLOGIES
MASTER 2 – CRYPTOLOGIE ET SÉCURITÉ INFORMATIQUE

DISCOVERING AND EXPLOITING VULNERABLE TUNNELLING HOSTS

Réalisé par :

DIALLO Mamadou Aliou & SADICK Youssouf Charaf

Encadrant :

M. Abdou Guermouche
`abdou.guermouche@labri.fr`
LABRI | Université de Bordeaux

ANNÉE UNIVERSITAIRE 2025–2026

Table des matières

1	Introduction	2
2	Protocoles de tunnelling existants	4
2.1	IPIP & IP6IP6	4
2.2	Generic Routing Encapsulation - GRE & GRE6	5
2.3	6in4	5
2.4	4in6	6
2.5	Generic UDP Encapsulation - GUE	7
3	Méthodes de découverte des hôtes de tunnelling vulnérables	8
3.1	Scan standard	8
3.2	Scan Echo/Reply	9
3.3	Scan TTL Expired	10
4	Menaces et vulnérabilités liées au tunnelling	12
4.1	Vulnérabilités des protocoles IPIP & IP6IP6	12
4.2	Vulnérabilités du protocole GRE	12
4.3	Vulnérabilités des protocoles 6in4 et 4in6	13
4.4	Vulnérabilités du protocole GUE	14
5	Attaques basées sur le tunnelling	16
5.1	Attaques par boucles de routage	16
5.2	Attaque par abus des signalements	17
5.3	Attaque Ping-Pong par amplification	17
5.4	Attaque Tunnelled Temporal Lensing (TuTL)	20
6	Contre-mesures et bonnes pratiques	26
6.1	Défenses au niveau hôte	26
6.2	Défenses au niveau réseau	27
7	Conclusion	30
8	Bibliographie	31

1 Introduction

Depuis plusieurs décennies, internet repose sur un ensemble de protocoles et de mécanismes permettant d'assurer la communication entre des réseaux de plus en plus nombreux, hétérogènes et géographiquement distribués. Parmi ces mécanismes, le tunnelling occupe une place centrale. Il permet d'encapsuler des paquets réseau à l'intérieur d'autres paquets afin de transporter des données à travers des infrastructures qui ne sont pas nativement compatibles. Cette technique est notamment utilisée pour interconnecter des réseaux isolés, assurer la transition entre IPv4 et IPv6, ou encore mettre en place des réseaux privés virtuels (VPN).

Les protocoles de tunnelling tels que IP-in-IP (IPIP), Generic Routing Encapsulation (GRE), IPv6 in IPv4 (6in4) ou IPv4 in IPv6 (4in6) sont aujourd'hui largement déployés dans de nombreux contextes : réseaux d'entreprise, fournisseurs de services internet, infrastructures cloud ou encore environnements de virtualisation. Leur popularité s'explique par leur simplicité, leur flexibilité et leur support natif dans de nombreux systèmes d'exploitation, notamment Linux. Ces protocoles ont souvent été conçus à une époque où internet était un environnement plus restreint, basé sur la confiance entre acteurs, ce qui explique certains choix de conception encore visibles aujourd'hui.

Toutefois, cette simplicité a un coût important en matière de sécurité. En effet, la majorité des protocoles de tunnelling ne proposent ni authentification ni chiffrement par défaut. Leur utilisation sécurisée repose donc sur des mécanismes externes, tels que IPsec, ou sur une configuration rigoureuse des équipements réseau. En pratique, ces mesures de protection ne sont pas systématiquement mises en œuvre. Les tunnels sont parfois laissés ouverts par défaut, acceptant du trafic provenant de sources arbitraires, ce qui peut transformer des hôtes légitimes en points d'entrée involontaires pour des attaquants.

Plusieurs travaux de recherche ont mis en évidence l'existence d'hôtes mal configurés, appelés hôtes de tunnelling ouverts, qui acceptent du trafic de tunnellation non authentifié depuis internet. Ces hôtes peuvent alors être exploités comme des proxies unidirectionnels, permettant à un attaquant de masquer l'origine réelle de ses paquets. Ce comportement est particulièrement problématique dans un contexte où de nombreux réseaux déploient des mécanismes de filtrage des adresses source afin de limiter l'usurpation d'adresses IP. L'existence de tels hôtes remet en question l'efficacité réelle de ces mécanismes de défense lorsqu'ils sont contournés indirectement.

Au-delà du simple contournement des politiques de filtrage, les hôtes de tunnelling vulnérables peuvent également faciliter des attaques plus complexes. Par exemple, un attaquant peut exploiter ces tunnels pour accéder à des réseaux privés, cartographier une infrastructure interne ou contourner des règles de pare-feu. Dans certains cas, il est même possible de provoquer des comportements inattendus au niveau administratif, comme l'envoi de faux rapports d'abus vers des organisations innocentes, ce qui constitue une forme de déni de service indirect.

Les attaques par déni de service (DoS) constituent un autre aspect critique de la problématique. Historiquement, les attaques DoS reposent principalement sur l'amplification du volume de trafic envoyé vers une cible. Cependant, des recherches récentes ont montré

que des protocoles réseau peuvent être détournés pour produire des effets d’amplification temporelle, où des paquets envoyés sur une longue période sont regroupés dans une fenêtre temporelle très courte à l’arrivée chez la victime. Ce type d’attaque, connu sous le nom de temporal lensing, permet d’augmenter considérablement l’impact d’une attaque sans nécessairement augmenter le volume total de trafic généré par l’attaquant.

Dans le contexte des protocoles de tunnelling, ces attaques prennent une dimension particulière. La capacité d’un hôte vulnérable à décapsuler et à retransmettre des paquets peut être exploitée pour créer des boucles de trafic, concentrer des flux dans le temps ou encore détourner de la bande passante sortante. Ces comportements peuvent conduire à des attaques de type Economic Denial of Sustainability (EDoS), particulièrement préoccupantes dans les environnements cloud où les coûts sont directement liés à la consommation de ressources réseau.

Malgré l’importance de ces enjeux, l’état de l’art montre que la sécurité des protocoles de tunnelling est encore insuffisamment étudiée de manière globale. De nombreux travaux se concentrent sur un protocole spécifique ou sur un type d’attaque isolé, sans considérer l’ensemble de l’écosystème, incluant IPv4, IPv6, les mécanismes de transition et les environnements modernes comme le cloud. De plus, l’identification à grande échelle des hôtes vulnérables sur internet reste un défi technique, en particulier pour IPv6, dont l’espace d’adressage rend les scans exhaustifs irréalistes.

Dans ce contexte, il apparaît nécessaire de mener une analyse approfondie des protocoles de tunnelling les plus couramment utilisés, de leurs faiblesses de sécurité et des risques associés à leur mauvaise configuration. Ce rapport s’inscrit dans cette perspective en proposant une étude des principales techniques de tunnelling, une analyse de leur exposition réelle sur internet et une exploration des attaques rendues possibles par ces vulnérabilités. Enfin, ce travail vise à discuter des contre-mesures existantes et des bonnes pratiques à adopter afin de renforcer la sécurité des infrastructures utilisant des mécanismes de tunnelling.

Ce rapport s’appuie principalement sur l’article « Haunted by Legacy : Discovering and Exploiting Vulnerable Tunnelling Hosts » de Beitis et Vanhoef[1]. Il en reprend les principaux résultats expérimentaux et propose une analyse structurée des protocoles étudiés, des méthodes de découverte employées, ainsi que des attaques rendues possibles par leur exposition non maîtrisée.

2 Protocoles de tunnelling existants

Le tunnelling est une technique réseau consistant à encapsuler un paquet d'un protocole à l'intérieur d'un autre paquet afin de permettre son transport à travers une infrastructure intermédiaire qui ne le supporte pas nativement. Ce mécanisme crée un lien logique entre deux points distants, donnant l'illusion d'une connectivité directe malgré les contraintes techniques ou administratives du réseau sous-jacent.

Le tunnelling joue aussi un rôle majeur dans la transition entre IPv4 et IPv6. Des mécanismes comme 6in4 ou 4in6 permettent d'encapsuler un protocole IP dans l'autre, facilitant l'interopérabilité entre réseaux hétérogènes. Dans les environnements modernes, en particulier les infrastructures cloud et les plateformes de virtualisation, il est devenu un outil central pour la mise en place de réseaux logiques indépendants de la topologie physique, notamment via des architectures overlay.

Dans les sous-sections suivantes, nous présentons les principaux protocoles de tunnelling en décrivant leur fonctionnement et leurs spécificités techniques :

2.1 IPIP & IP6IP6

Le protocole IP-*in*-IP (IPIP) ou (IP6IP6 pour les adresses IPV6) est l'un des mécanismes de tunnelling les plus simples et les plus anciens définis pour les réseaux IP. Standardisé dans la RFC 2003, il repose sur un principe fondamental : l'encapsulation d'un paquet IP à l'intérieur d'un autre paquet IP. Concrètement, le paquet original est conservé intégralement et devient la charge utile d'un nouveau paquet IP, auquel est ajouté un en-tête IP externe permettant son acheminement à travers le réseau intermédiaire. Cette approche permet de transporter du trafic IP à travers des infrastructures qui ne disposent pas nécessairement d'une connectivité directe entre la source et la destination finales.[2]

Le protocole IPIP a historiquement été conçu pour répondre à des besoins simples d'interconnexion entre réseaux distants, notamment dans des contextes où la mise en place de solutions plus complexes était difficile ou inutile. Il a ainsi été utilisé pour relier des sous-réseaux isolés, mettre en œuvre des mécanismes de transition entre différentes versions du protocole IP, ou encore faciliter certaines architectures de routage spécifiques. Sa conception minimaliste présente l'avantage d'être largement supportée par les équipements réseau et les systèmes d'exploitation modernes, en particulier les noyaux Linux, où l'activation d'un tunnel IPIP ne nécessite qu'une configuration limitée.

D'un point de vue fonctionnel, le comportement d'un tunnel IPIP(ou IP6IP6) est relativement simple. Lorsqu'un paquet encapsulé est reçu, l'équipement terminal du tunnel retire l'en-tête IP externe et transmet le paquet interne selon sa table de routage locale. Ce processus de décapsulation est transparent pour le paquet transporté, qui est traité comme s'il avait été directement reçu par l'équipement. Cette transparence constitue à la fois la force et la faiblesse du protocole, car elle permet une grande flexibilité tout en introduisant des risques lorsque le tunnel est mal configuré ou exposé à des sources non maîtrisées[3].

Il est important de souligner que le protocole IPIP ne prévoit aucun mécanisme natif de sécurisation du trafic. Contrairement à des solutions telles qu'IPsec, il ne fournit ni chiffrement, ni authentification, ni intégrité des données. La sécurité du tunnel repose donc entièrement sur des mécanismes externes ou sur une configuration stricte des équipements

réseau, par exemple via des règles de filtrage ou des restrictions explicites des adresses autorisées à utiliser le tunnel. Dans les environnements modernes, où les infrastructures sont souvent complexes et dynamiques, ces exigences de configuration ne sont pas toujours respectées, ce qui peut conduire à des situations d'exposition involontaire[1].

2.2 Generic Routing Encapsulation - GRE & GRE6

Le protocole **Generic Routing Encapsulation** (GRE ou GRE6) propose une approche plus générale et plus flexible du tunnelling que IPIP. Standardisé initialement dans la RFC 2784, GRE introduit un mécanisme d'encapsulation générique permettant de transporter différents types de protocoles de couche réseau au sein d'un paquet IP. Contrairement à IPIP, qui se limite à l'encapsulation d'un paquet IP dans un autre paquet IP, GRE ajoute un en-tête spécifique entre l'en-tête IP externe et le paquet encapsulé. Cet en-tête GRE permet notamment d'indiquer le type de protocole encapsulé, rendant possible le transport de paquets IPv4, IPv6 ou d'autres protocoles identifiés par un champ `EtherType`[4].

Cette capacité à encapsuler des protocoles hétérogènes a fait de GRE une solution largement adoptée pour la mise en œuvre de tunnels entre réseaux distants, pour le transport de trafic multi-protocoles, ainsi que pour certaines architectures de virtualisation et de routage avancées. GRE est également utilisé comme brique de base dans des mécanismes plus complexes, notamment lorsqu'il est combiné à des solutions de sécurisation externes. Comme IPIP, il est nativement supporté par de nombreux équipements réseau et systèmes d'exploitation, ce qui facilite son déploiement dans des environnements variés.

Le format de l'en-tête GRE a été étendu dans la RFC 2890 afin d'introduire des champs optionnels, parmi lesquels un champ `Key` de 4 octets. Ce champ permet d'associer un identifiant logique au tunnel et de filtrer les paquets entrants : un hôte ne traitera un paquet GRE que si la valeur de la clé correspond à celle attendue. Bien que ce mécanisme puisse compliquer la génération de paquets valides par un adversaire passif ou opportuniste, il ne constitue pas une véritable mesure de sécurité. En effet, la clé GRE n'est ni chiffrée ni protégée, et peut être facilement interceptée par un attaquant capable d'observer le trafic réseau. De plus, cette clé reste généralement statique, sauf modification explicite de la configuration, ce qui limite fortement son efficacité en pratique[1].

D'un point de vue sécuritaire, GRE présente des limitations similaires à celles de IPIP. Le protocole ne fournit aucun mécanisme natif de chiffrement, d'authentification forte ou de protection de l'intégrité des données. Un attaquant positionné en homme-du-milieu peut ainsi injecter ou modifier des paquets GRE, tandis qu'un attaquant en homme-sur-le-côté peut, dans certains cas, exploiter l'absence de protections pour construire des paquets malveillants, en particulier lorsque le champ `Key` n'est pas utilisé. Pour ces raisons, GRE est généralement déployé conjointement avec des mécanismes de sécurité supplémentaires, tels qu'IPsec, afin de garantir la confidentialité et l'authenticité du trafic encapsulé.

2.3 6in4

Le mécanisme **6in4** désigne l'encapsulation d'un paquet IPv6 à l'intérieur d'un paquet IPv4. Défini dans la RFC 4213, ce protocole a été conçu dans le contexte de la transition progressive d'IPv4 vers IPv6, afin de permettre le transport de trafic IPv6 à travers des

infrastructures ne supportant que le protocole IPv4. Dans ce schéma, le paquet IPv6 original est conservé intégralement et encapsulé comme charge utile d'un paquet IPv4, dont l'en-tête externe assure l'acheminement à travers le réseau intermédiaire. Le fonctionnement du protocole **6in4** repose sur un principe simple de décapsulation aux extrémités du tunnel. À la réception du datagramme, l'équipement terminal retire l'en-tête IPv4 externe et transmet le paquet IPv6 interne à la pile réseau locale, où il est traité de manière transparente. Ce mécanisme permet ainsi à des interfaces ne disposant pas d'une connectivité IPv6 native de communiquer avec des hôtes IPv6 distants, sans nécessiter de modifications profondes de l'infrastructure existante[1].

Un cas particulier largement associé au protocole **6in4** est le mécanisme de transition **6to4**. Contrairement à un tunnel **6in4** classique, **6to4** introduit un schéma d'adressage IPv6 spécifique, dérivé directement de l'adresse IPv4 de l'hôte. Une adresse **6to4** prend la forme `2002:IPV4_ADDRESS_IN_HEX::`, où l'adresse IPv4 est convertie en notation hexadécimale. Par exemple, pour une adresse IPv4 `a.b.c.d`, l'adresse **6to4** correspondante sera `2002:a.b.c.d::`. Dans la suite de ce rapport, le terme **6in4** désignera de manière générique l'encapsulation d'IPv6 dans IPv4, tandis que le terme **6to4** sera réservé aux adresses suivant ce format spécifique[1, 5, 6].

D'un point de vue sécuritaire, **6in4** présente des limitations similaires à celles des autres protocoles de tunnelling simples. Il ne fournit aucun mécanisme natif de chiffrement, d'authentification ou de contrôle d'intégrité, ce qui implique que la sécurité du tunnel repose exclusivement sur des mécanismes externes ou sur une configuration stricte des équipements réseau. En l'absence de telles mesures, le protocole peut être exposé à des attaques par injection ou par usurpation de paquets encapsulés.

2.4 4in6

Le protocole **4in6** correspond au mécanisme inverse de **6in4**, à savoir l'encapsulation d'un paquet IPv4 à l'intérieur d'un paquet IPv6. Défini dans la RFC 2473, il a été introduit pour permettre le transport de trafic IPv4 à travers des réseaux reposant majoritairement, voire exclusivement, sur IPv6. Ce mécanisme vise à assurer l'interopérabilité entre les deux versions du protocole IP pendant la phase de transition, en particulier dans des environnements où certaines applications ou équipements ne supportent pas encore IPv6[1].

Comme pour les autres techniques de tunnelling par encapsulation, le fonctionnement de **4in6** est transparent pour le paquet transporté. À la réception du datagramme encapsulé, l'en-tête IPv6 externe est supprimé, et le paquet IPv4 interne est transmis à la couche réseau locale pour être traité selon les règles de routage habituelles. Cette approche permet de maintenir la connectivité IPv4 au sein d'infrastructures IPv6, sans nécessiter de mécanismes de traduction d'adresses ou de modification du trafic applicatif.

Cependant, à l'instar de IPIP et **6in4**, le protocole **4in6** ne propose aucun mécanisme de sécurité intrinsèque. L'absence de chiffrement et d'authentification rend le tunnel vulnérable à l'observation, à la modification ou à l'injection de paquets par un attaquant positionné sur le chemin réseau. En pratique, l'utilisation de **4in6** dans des environnements non maîtrisés nécessite donc l'ajout de solutions de sécurisation complémentaires afin de garantir la confidentialité et l'intégrité des communications.

2.5 Generic UDP Encapsulation - GUE

La Generic UDP Encapsulation (GUE) est un mécanisme de tunnelling reposant sur l'encapsulation de paquets réseau arbitraires à l'intérieur de datagrammes UDP. Contrairement aux protocoles de tunnelling plus anciens, tels que IPIP ou GRE, GUE s'appuie explicitement sur UDP afin de tirer parti des optimisations largement supportées par les infrastructures réseau modernes, notamment dans les centres de données et les environnements virtualisés. Cette approche reflète une évolution des pratiques de conception des protocoles, où la compatibilité avec les équipements intermédiaires et les mécanismes d'optimisation du trafic est devenue un objectif central.

Le principe général de GUE consiste à encapsuler un paquet réseau complet pouvant appartenir à différentes couches du modèle OSI dans la charge utile d'un paquet UDP. À la différence de mécanismes minimalistes comme Foo-over-UDP, GUE introduit un en-tête d'encapsulation dédié, inséré immédiatement après l'en-tête UDP. Cet en-tête permet d'indiquer explicitement le type de protocole encapsulé et de transporter des informations supplémentaires optionnelles. Cette conception rend GUE à la fois générique, extensible et adaptable à des cas d'usage variés[7].

L'utilisation d'UDP comme protocole de transport externe confère plusieurs avantages techniques. D'une part, elle permet au trafic encapsulé de bénéficier des mécanismes de répartition de charge présents dans les réseaux modernes, tels que l'ECMP ou le **Receive Side Scaling** (RSS), qui reposent souvent sur les champs du tuple UDP/IP. D'autre part, le support quasi universel des optimisations matérielles pour UDP, comme le calcul de checksum ou la segmentation hors charge (offload), contribue à réduire le coût de traitement des paquets encapsulés sur les hôtes. Ces propriétés expliquent pourquoi GUE et, plus largement, l'encapsulation UDP sont devenues des briques fondamentales des solutions de virtualisation réseau contemporaines[7].

Un autre aspect clé de GUE réside dans son caractère extensible. L'en-tête GUE prévoit la possibilité d'ajouter des champs optionnels, permettant par exemple de transporter des identifiants de réseau virtuel, des métadonnées de contrôle ou des informations liées à l'optimisation des performances. Cette extensibilité offre une grande liberté aux implémentations, mais introduit également une complexité supplémentaire par rapport aux protocoles de tunnelling plus simples. En pratique, GUE est principalement utilisé dans des contextes contrôlés, tels que les infrastructures cloud ou les réseaux de centres de données, où les extrémités du tunnel sont connues et administrées de manière centralisée.

3 Méthodes de découverte des hôtes de tunnelling vulnérables

L'identification des hôtes de tunnelling vulnérables c'est-à-dire ceux qui acceptent et décapsulent du trafic encapsulé provenant de sources arbitraires sans authentification repose sur sept (7) techniques de scan distinctes. Ces méthodes se répartissent en trois grandes familles : **les scans standards**, **les scans Echo/Reply** et **les scans TTL Expired**. Chacune est déclinée en variantes selon le protocole ciblé et les mécanismes de filtrage à contourner. Elles exploitent toutes le même principe : l'hôte cible reçoit un paquet encapsulé, le décapsule, et son traitement du paquet interne génère une réponse observable par le scanner.

Trois comportements permettent de caractériser cette vulnérabilité :

- **Forwarding externe** : l'hôte décapsule le paquet et le retransmet vers un tiers, agissant comme un proxy unidirectionnel.
- **Traitement local** : l'hôte décapsule le paquet et traite localement la charge utile (exemple : réponse ICMP Echo).
- **Tentative de forwarding détectable** : l'hôte décapsule le paquet et tente de le transmettre, ce qui peut être observé indirectement via un message ICMP Time Exceeded.

La présence de l'un de ces comportements suffit à démontrer l'existence d'une terminaison de tunnel accessible publiquement.

3.1 Scan standard

Cette famille regroupe toutes les techniques où le scanner attend le retour d'un paquet décapsulé et forwardé par la cible. Elle inclut le scan standard de base, ses variantes par usurpation, et les scans spécifiques aux protocoles de transition.

Scan standard

Cette méthode vise à détecter les hôtes qui acceptent et retransmettent le trafic tunnelé. Le scanner envoie un paquet dont l'en-tête externe a pour destination la cible, tandis que l'en-tête interne a pour source la cible elle-même et pour destination le scanner. Si le scanner reçoit le paquet interne, cela signifie que la cible a accepté le paquet externe, l'a décapsulé, et a retransmis le contenu. L'hôte agit alors comme un proxy unidirectionnel[1].

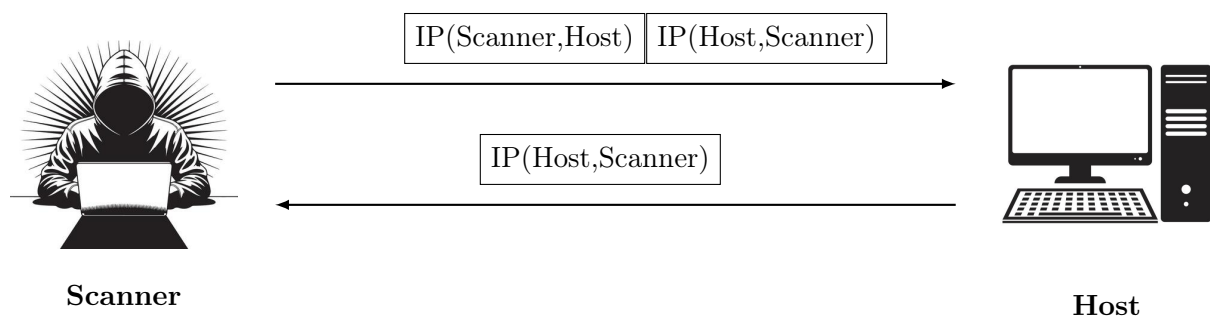


FIGURE 1 – Scan standard avec IPIP.

Scan par usurpation de sous-réseau

Cette technique permet de contourner les filtres qui rejettent les adresses totalement étrangères mais acceptent celles du même sous-réseau. Le principe est identique au scan standard, à la différence près que l'adresse source du paquet interne appartient au même bloc d'adresses que la cible (même /24 en IPv4, même /112 en IPv6). Certains systèmes autonomes filtrent les paquets sortants dont l'adresse source ne leur appartient pas, mais tolèrent les adresses de leur propre plage. Ce scan révèle ces filtrages partiels.

Scan par usurpation complète

Cette méthode identifie les réseaux qui n'implémentent aucun filtrage source, en violation des recommandations RFC 2827 et 3704 (BCP38). L'adresse source du paquet interne est totalement arbitraire : elle peut être celle d'un serveur DNS public comme 8.8.8.8, ou toute autre adresse choisie aléatoirement. Si le paquet parvient au scanner, cela signifie que la cible peut forcer son réseau à émettre un paquet avec n'importe quelle adresse source. Ces hôtes, qualifiés de *spoofing-capable*, sont particulièrement dangereux.

Scan 6to4

Cette méthode cible spécifiquement les hôtes configurés en 6in4 (IPv6 dans IPv4) utilisant l'adressage automatique 6to4. L'en-tête externe est IPv4, l'en-tête interne est IPv6. L'adresse IPv6 source est dérivée de l'adresse IPv4 de la cible selon le format 2002:IPv4_en_hexa::. Si l'hôte accepte ce paquet, le décapsule et forward le paquet IPv6 interne, il se comporte comme un relais 6to4 ouvert. Ces relais sont extrêmement répandus : l'étude en a détecté plus de 2,1 millions, soit près de la moitié des vulnérabilités totales.

Scan IPv4-mapped

Cette variante du scan 6in4 utilise le format d'adresse IPv4-mappée ::ffff:IPv4_EN_HEX. Certains systèmes ne répondent qu'à ce format spécifique d'adresse IPv6 dérivée d'une IPv4. Cette technique complète donc la détection des relais de transition IPv6 en couvrant un ensemble plus large de configurations.[1]

3.2 Scan Echo/Reply

Cette méthode détecte les hôtes qui acceptent et décapsulent le trafic tunnelé mais ne le forwardent pas vers l'extérieur, que ce soit pour des raisons de politique interne, de configuration firewall, ou d'absence de routage. Le paquet interne est une requête ICMP Echo à destination de la cible elle-même. L'hôte reçoit le paquet externe, le décapsule, traite localement la requête ICMP qui lui est adressée, et y répond par une Echo Reply à destination du scanner. Même si l'hôte ne retransmet aucun trafic vers des tiers, cette réponse prouve que le tunnel existe et est accessible depuis l'extérieur.[1]

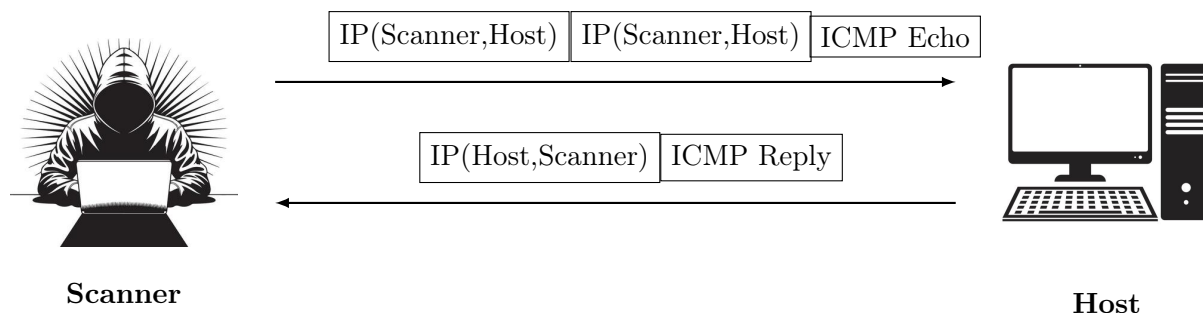


FIGURE 2 – Scan Echo/Reply avec IPIP.

3.3 Scan TTL Expired

Cette technique permet de contourner les **NAT**, les **réseaux privés** et les **firewalls stricts** qui empêcheraient le retour direct de paquets décapsulés. Le paquet interne est construit avec un TTL (IPv4) ou un Hop Limit (IPv6) égal à 1, et une destination quelconque différente de la cible. Lorsque l'hôte décapsule ce paquet et tente de le forwarder, le TTL expire immédiatement. Il génère alors un message ICMP Time Exceeded à destination de l'adresse source du paquet interne, qui est celle du scanner. Ce message d'erreur, qui contient l'en-tête du paquet ayant expiré, prouve que l'hôte a bien reçu, décapsulé et tenté de "forwarder" le paquet tunnelé. Même derrière un NAT, la vulnérabilité est détectable.[1]

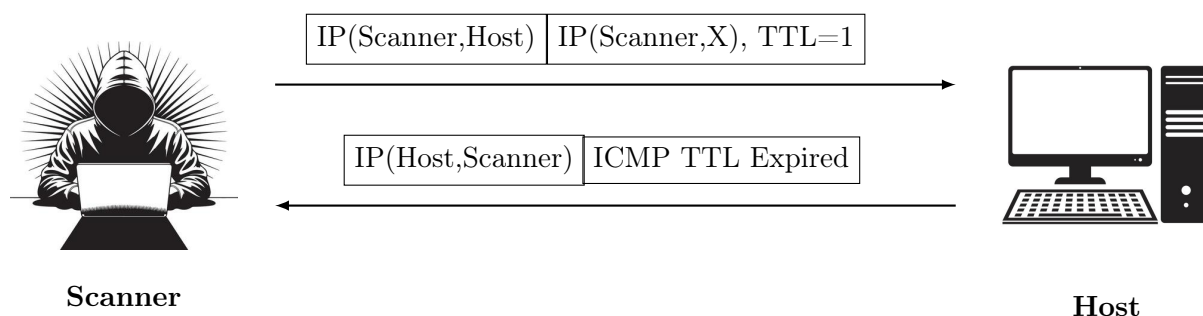


FIGURE 3 – Scan TTL expiré avec IPIP.

Synthèse des méthodes par protocole

Toutes les méthodes ne s'appliquent pas à tous les protocoles. Certains protocoles, comme 4in6, sont particulièrement difficiles à scanner car l'adresse IPv4 de la cible n'est pas connue depuis l'en-tête IPv6. D'autres, comme GUE, n'ont révélé aucun hôte vulnérable lors des tests préliminaires réalisés dans [1] et ont été exclus des scans à grande échelle.

Le tableau 1 présente, pour chaque protocole, les méthodes de scan applicables telles qu'elles ont été mises en œuvre dans l'étude.

TABLE 1 – Couverture des scans par protocole et méthodologie

Méthodologie	IPv4	IPv6	GRE	GRE6	4in6	6in4	GUE
Standard	✓	✓	✓	✓	✓	✓	✓
Écho/Reply	✓	✓	✓	✓	–	⊗	×
TTL expired	✓	✓	✓	✓	✓	✓	×

✓ Scan réalisé dans cette étude[1].
 × Scan non réalisé dans cette étude.
 ⊗ Scan réalisé dans d'autres travaux[11, 12].
 – Méthode non applicable à ce protocole.

4 Menaces et vulnérabilités liées au tunnelling

4.1 Vulnérabilités des protocoles IPIP & IP6IP6

Les faiblesses de sécurité du protocole IPIP découlent directement de ses choix de conception initiaux. En l'absence de mécanismes d'authentification, tout paquet IPIP reçu par un équipement configuré pour accepter ce type de trafic peut être traité comme légitime, indépendamment de l'identité réelle de l'émetteur. Cette caractéristique pose un problème majeur lorsque le tunnel est accessible depuis internet ou depuis un réseau non fiable, car elle ouvre la voie à des usages détournés du protocole.

Un équipement utilisant IPIP peut devenir vulnérable lorsqu'il accepte des paquets encapsulés provenant de sources arbitraires et destinés à des adresses internes ou externes non explicitement autorisées. Dans ce cas, l'équipement agit involontairement comme un relais réseau. Il reçoit le paquet encapsulé, en extrait le contenu, puis transmet le paquet interne sans vérifier si ce comportement correspond à une utilisation légitime du tunnel. Cette capacité à relayer du trafic de manière transparente peut être exploitée par un attaquant pour masquer l'origine réelle de ses communications ou pour contourner des politiques de sécurité mises en place sur le réseau. L'un des aspects particulièrement problématiques de cette vulnérabilité réside dans son interaction avec les mécanismes de filtrage des adresses source. De nombreux réseaux déploient des contre-mesures visant à empêcher l'usurpation d'adresses IP, telles que le filtrage en sortie ou les bonnes pratiques recommandées par la communauté internet. Toutefois, dans le cas d'un tunnel IPIP mal configuré, le paquet interne peut apparaître comme légitime du point de vue du réseau intermédiaire, car il semble provenir de l'équipement qui effectue la décapsulation. Ainsi, même en présence de mécanismes anti-spoofing, un attaquant peut parvenir à injecter du trafic falsifié en exploitant le comportement du tunnel.

Ces faiblesses peuvent avoir des conséquences variées et parfois sévères. Elles permettent notamment la réalisation d'attaques par déni de service, où un équipement vulnérable est utilisé pour relayer ou amplifier du trafic vers une cible. Elles peuvent également conduire au contournement de règles de pare-feu ou de listes de contrôle d'accès, offrant à un attaquant un accès indirect à des ressources normalement protégées. Dans certains scénarios, l'exploitation d'un tunnel IPIP vulnérable peut même permettre la collecte d'informations sensibles ou la reconnaissance d'une infrastructure interne, constituant ainsi une étape préalable à des attaques plus élaborées.

Enfin, la simplicité d'exploitation de ces vulnérabilités rend leur détection particulièrement difficile. Le trafic IPIP peut apparaître comme légitime au niveau des équipements intermédiaires, et les comportements anormaux ne sont pas toujours visibles sans mécanismes de supervision ou de détection spécifiques. Cette discrétion, combinée à la fréquence des configurations par défaut insuffisamment sécurisées, explique pourquoi IPIP reste aujourd'hui un vecteur d'attaque pertinent malgré son ancienneté.[1]

4.2 Vulnérabilités du protocole GRE

Le protocole **Generic Routing Encapsulation** (GRE) hérite de nombreuses faiblesses de sécurité structurelles similaires à celles d'IPIP. Bien qu'il introduise un en-tête supplémentaire permettant l'encapsulation de protocoles hétérogènes, GRE ne fournit par défaut aucun mécanisme de chiffrement, d'authentification forte ou de contrôle d'intégrité.

En conséquence, un hôte configuré de manière permissive peut accepter des paquets GRE provenant de sources arbitraires et les décapsuler sans vérifier leur légitimité.

Une configuration particulièrement problématique est celle des tunnels GRE multi-points (*mGRE*), où le champ `remote` est laissé vide. Dans ce cas, l'hôte accepte des paquets GRE de n'importe quelle adresse source. Les travaux récents montrent que cette configuration est largement déployée sur internet et constitue une source majeure de vulnérabilités. Un attaquant peut alors exploiter un hôte GRE vulnérable comme un proxy unidirectionnel, de manière analogue aux attaques observées sur `IPIP`, en injectant des paquets encapsulés dont l'en-tête interne possède une adresse source usurpée.

Le champ optionnel `Key`, introduit par la RFC 2890, est parfois présenté comme un mécanisme de protection. Toutefois, cette clé n'est ni chiffrée ni dynamique. Un attaquant capable d'observer le trafic peut facilement la récupérer, et même en l'absence d'observation directe, une attaque par force brute reste envisageable. En pratique, ce champ ne constitue donc pas une barrière de sécurité fiable, mais tout au plus un obstacle mineur pour des attaques opportunistes.

Les conséquences de ces vulnérabilités sont significatives. Des études à grande échelle ont révélé l'existence de plus d'un million d'hôtes GRE vulnérables acceptant du trafic non authentifié. Ces hôtes peuvent être utilisés pour contourner des mécanismes de filtrage d'adresses source, mener des attaques par déni de service, ou encore participer à des attaques d'amplification et de concentration temporelle du trafic. Dans certains cas, les hôtes GRE vulnérables sont intégrés à des infrastructures critiques, telles que des réseaux d'opérateurs mobiles ou des routeurs BGP, ce qui accroît encore l'impact potentiel de leur exploitation[1]

4.3 Vulnérabilités des protocoles `6in4` et `4in6`

Comme présenté en section 2.3, le mécanisme `6in4` repose sur l'encapsulation transparente d'un paquet IPv6 dans un paquet IPv4, tandis que la section 2.4 a décrit `4in6`, qui réalise l'opération inverse. Dans les deux cas, le principe de fonctionnement est fondé sur une encapsulation simple suivie d'une décapsulation aux extrémités du tunnel, sans mécanisme de protection intrinsèque.

Ces caractéristiques structurelles constituent la principale source de vulnérabilité de ces protocoles. En l'absence d'authentification, de chiffrement ou de contrôle d'intégrité natif, un hôte configuré pour accepter du trafic `6in4` ou `4in6` peut décapsuler des paquets provenant de sources arbitraires. Un attaquant peut ainsi envoyer des paquets encapsulés vers un hôte vulnérable, qui transmettra ensuite le paquet interne selon sa table de routage locale. Le trafic décapsulé apparaît alors comme émis par l'hôte tunnel, ce qui permet de contourner certains mécanismes de filtrage anti-usurpation déployés dans les réseaux IPv4 ou IPv6.

Dans le cas particulier de `6in4`, l'utilisation du mécanisme `6to4` (décrit en section 2.3) peut faciliter l'exploitation. Le schéma d'adressage déterministe basé sur l'adresse IPv4 simplifie la génération de paquets IPv6 plausibles par un attaquant, sans connaissance détaillée de l'architecture interne. Des études de mesure à grande échelle ont montré qu'un nombre significatif d'hôtes exposent encore des terminaisons `6in4` accessibles publiquement, élargissant la surface d'attaque.

Le protocole `4in6`, bien que généralement moins répandu, présente des risques analogues. Un hôte mal configuré peut être utilisé comme relais pour émettre du trafic IPv4 dont l'adresse source interne est falsifiée. Ce comportement fragilise les mécanismes de

filtrage dans les environnements hybrides IPv4/IPv6, où les politiques de sécurité ne prennent pas toujours en compte le trafic encapsulé.

Enfin, certaines implémentations de ces mécanismes ont été associées à des comportements susceptibles d'amplifier l'impact d'attaques par déni de service, notamment en cas de bouclage de tunnels ou de retransmissions répétées de paquets encapsulés. Dans des infrastructures cloud ou multi-tenant, où la facturation peut dépendre du volume de trafic sortant, ce type d'exploitation peut entraîner à la fois une saturation des ressources réseau et des coûts économiques significatifs.

Ainsi, bien que 6in4 et 4in6 répondent à des besoins légitimes de transition entre IPv4 et IPv6, leur déploiement sans mécanismes de sécurisation complémentaires expose les infrastructures à des risques comparables à ceux observés pour d'autres protocoles de tunnelling non authentifiés.[1]

4.4 Vulnérabilités du protocole GUE

Bien que GUE ait été conçu pour répondre aux exigences des réseaux modernes en matière de performance et de flexibilité, il hérite de plusieurs faiblesses structurelles communes aux protocoles de tunnelling. En premier lieu, GUE ne fournit pas nativement de mécanismes de chiffrement ou d'authentification forte. Comme pour IPIP ou GRE, la sécurité du tunnel repose donc sur des mécanismes externes ou sur une configuration stricte des équipements réseau. En l'absence de telles mesures, un hôte acceptant du trafic GUE peut traiter des paquets encapsulés provenant de sources non autorisées.

Le caractère générique de GUE constitue à la fois une force et une faiblesse. La possibilité d'encapsuler pratiquement n'importe quel protocole IP rend le tunnel extrêmement flexible, mais complique également l'application de politiques de filtrage efficaces. Un pare-feu ou un dispositif de sécurité intermédiaire peut être incapable d'inspecter le contenu réel du trafic encapsulé, se contentant d'analyser les en-têtes UDP externes. Cette opacité peut être exploitée pour faire transiter des flux normalement interdits ou inattendus, contournant ainsi les règles de sécurité basées sur l'inspection des protocoles internes.

Par ailleurs, l'utilisation d'UDP comme transport externe facilite l'intégration du trafic encapsulé dans les mécanismes de répartition de charge du réseau. Si cet aspect est bénéfique du point de vue des performances, il peut également être détourné à des fins malveillantes. Un attaquant peut exploiter un tunnel GUE mal configuré pour générer un trafic important ou déséquilibré, conduisant à une saturation des ressources réseau ou à une dégradation des performances des systèmes cibles. Dans certains scénarios, cette capacité peut être utilisée pour amplifier des attaques par déni de service, notamment dans des environnements mutualisés.

La gestion des en-têtes optionnels de GUE introduit également des risques spécifiques. La présence de champs extensibles accroît la surface d'attaque du protocole, en particulier si les implémentations ne valident pas rigoureusement la structure et la cohérence des en-têtes reçus. Des paquets malformés ou volontairement construits de manière atypique peuvent provoquer des comportements inattendus, allant de la simple perte de paquets à des dysfonctionnements plus graves au niveau du plan de contrôle ou du plan de données. La complexité accrue de GUE rend sa surveillance plus délicate. L'identification d'abus ou de comportements anormaux nécessite des outils capables d'analyser non seulement

le trafic UDP, mais également les en-têtes **GUE** et les protocoles encapsulés. En l'absence de règles de détection adaptées, un tunnel **GUE** vulnérable peut rester exploité sur de longues périodes sans être détecté, ce qui est particulièrement préoccupant dans des infrastructures de grande échelle telles que les centres de données ou les environnements cloud.[1]

5 Attaques basées sur le tunnelling

Cette section présente une modélisation des principales attaques rendues possibles par des hôtes de tunneling mal configurés ou vulnérables. Ces attaques exploitent les propriétés intrinsèques des mécanismes de tunneling, en particulier l'absence d'authentification, de contrôle d'état et de validation stricte des paquets encapsulés.

Nous nous concentrons sur quatre grandes classes d'attaques. Les deux premières relèvent du déni de service et de ses variantes : l'attaque Ping-Pong par amplification, qui exploite des boucles de décapsulation pour amplifier le trafic, et l'attaque *Tunnelled Temporal Lensing* (TuTL), qui permet de concentrer le trafic dans le temps afin de créer des pics soudains de charge. Les deux autres attaques illustrent des usages détournés du tunneling à des fins de contournement et d'abus : l'abus de mécanismes de signalement et de gestion des incidents, ainsi que la création de boucles de routage logiques permettant de détourner ou de perturber le trafic réseau.

Pour chaque attaque, nous décrivons le modèle de menace, le scénario d'exploitation et les mécanismes fondamentaux expliquant son efficacité, afin de mettre en évidence les risques systémiques associés à une exposition non maîtrisée des protocoles de tunneling sur internet.

5.1 Attaques par boucles de routage

L'attaque de type *Routing Loop DoS* repose sur un comportement observé lors de l'utilisation de tunnels IPv6 sur IPv4 (6in4) dans la pile réseau Linux. Cette attaque exploite la manière dont certains paquets IPv6 encapsulés sont décapsulés puis routés lorsqu'ils possèdent une adresse de destination construite de façon spécifique.

Le modèle d'attaque consiste à envoyer un paquet IPv6 encapsulé dans IPv4 vers un hôte disposant d'une interface 6in4 active. Le paquet IPv6 interne utilise une adresse de destination particulière, composée d'une séquence de zéros suivie de l'encodage hexadécimal de l'adresse IPv4 de l'hôte cible (par exemple : `::IPv4_ADDRESS_EN_HEX`). Cette structure d'adresse influence directement la décision de routage prise par le noyau après la décapsulation du paquet. Lors de la réception du paquet, le système vulnérable procède à la décapsulation du paquet IPv6 interne, puis consulte sa table de routage afin de déterminer l'interface de sortie. En raison de la forme spécifique de l'adresse de destination IPv6, le paquet est réinjecté vers l'interface de tunnelling 6in4, au lieu d'être transmis vers une interface de sortie classique. Le paquet est alors à nouveau encapsulé et traité comme un nouveau paquet entrant. Ce mécanisme engendre une boucle de routage locale au sein de l'hôte vulnérable. Le paquet est traité de manière répétée par la pile réseau, sans jamais quitter le système. À chaque passage dans la boucle, le champ `Hop Limit` du paquet IPv6 est décrémenté. Étant donné que ce champ est codé sur 8 bits, le paquet peut être traité jusqu'à 256 fois avant d'être supprimé lorsque la valeur atteint zéro[1].

L'impact de cette attaque est principalement lié à la consommation de ressources qu'elle provoque. Chaque itération de la boucle mobilise des ressources CPU pour le traitement des en-têtes, ainsi que de la bande passante sur l'interface de tunnelling. Un attaquant peut ainsi provoquer une dégradation significative des performances du système ciblé en envoyant un nombre limité de paquets soigneusement construits, sans recourir à un volume massif de trafic.

Cette attaque met en évidence les risques associés aux mécanismes de transition IPv4/IPv6 lorsqu'ils sont activés sans contrôle strict. Elle illustre également comment des interactions subtiles entre adressage IPv6 et logique de routage peuvent conduire à des comportements exploitables, soulignant la nécessité de restreindre l'utilisation des interfaces de tunnelling aux seuls cas où elles sont réellement nécessaires.

5.2 Attaque par abus des signalements

Cette attaque repose sur l'exploitation indirecte des mécanismes administratifs de gestion des abus déployés par de nombreux fournisseurs de services et d'hébergement. Contrairement aux attaques par déni de service classiques, elle ne vise pas directement les ressources techniques de la victime, mais s'appuie sur des processus humains ou automatisés déclenchés en réponse à un trafic perçu comme malveillant.

Le modèle d'attaque repose sur l'existence d'hôtes de tunnelling vulnérables permettant l'usurpation de l'adresse source des paquets transmis. En exploitant ces hôtes, un attaquant est en mesure de générer du trafic arbitraire qui semble provenir de la victime, sans que celle-ci n'ait réellement émis les paquets concernés. Ce trafic usurpé peut alors être dirigé vers des systèmes ou des organisations connues pour signaler rapidement les comportements suspects via des rapports d'abus.

Le scénario d'attaque peut être décrit comme suit. L'attaquant envoie des paquets encapsulés vers un hôte de tunnelling vulnérable, en construisant le paquet interne de manière à ce que l'adresse source apparente corresponde à celle de la victime. Une fois décapsulé, le trafic est transmis vers une cible tierce, par exemple un serveur surveillant activement les scans réseau ou les tentatives de connexion suspectes. Ce serveur génère alors automatiquement un signalement d'abus, attribuant l'activité malveillante à l'adresse IP usurpée, c'est-à-dire celle de la victime.

En répétant cette opération vers plusieurs organisations distinctes, l'attaquant peut provoquer une accumulation rapide de signalements d'abus à l'encontre de la victime. Ces rapports sont ensuite transmis aux fournisseurs d'accès, aux hébergeurs ou aux administrateurs système responsables de l'adresse IP concernée. Même en l'absence de trafic réellement émis par la victime, celle-ci se retrouve associée à des comportements malveillants qu'elle n'a pas produits.

L'impact de cette attaque est principalement de nature administrative. Un volume important de rapports d'abus peut entraîner une surcharge de travail pour les équipes techniques de la victime, contraintes d'analyser et de justifier un trafic qu'elles n'ont pas généré. Dans certains cas, ces signalements peuvent conduire à des mesures plus sévères, telles que la suspension temporaire d'un service, la résiliation d'un compte d'hébergement ou le blocage de l'adresse IP par des mécanismes de filtrage automatique. Lorsque la victime ne réside pas dans un environnement cloud, ces blocages peuvent également prendre la forme d'une mise sur liste noire, limitant fortement l'accès de l'hôte à certaines parties du réseau internet.

5.3 Attaque Ping-Pong par amplification

Modèle d'attaque

Nous considérons un attaquant externe ayant identifié au moins deux hôtes vulnérables, notés *A* et *B*, qui acceptent du trafic de tunnelling non authentifié (par exemple

IPIP ou IP6IP6). Ces hôtes décapsulent les paquets reçus et réémettent les paquets internes sans authentifier leur origine ni vérifier la cohérence du trafic.

Hypothèses initiales :

- Les hôtes A et B sont accessibles depuis internet.
- Les interfaces de tunnelling acceptent du trafic provenant de sources arbitraires.
- Les hôtes ne maintiennent pas d'état permettant de détecter ou de bloquer les boucles de décapsulation.

Scénario d'attaque

L'attaquant construit un unique paquet de taille maximale autorisée par le chemin réseau (typiquement le Maximum Transmission Unit - MTU), contenant plusieurs en-têtes IP emboîtés. Les adresses source et destination alternent à chaque encapsulation :

- En-tête i : source A , destination B .
- En-tête $i + 1$: source B , destination A .

Le paquet ainsi construit est envoyé à l'hôte A . Celui-ci décapsule le premier en-tête IP externe, révélant le paquet interne, puis le transmet vers B . L'hôte B répète le même processus et renvoie le paquet vers A . Cette alternance se poursuit jusqu'à ce que tous les en-têtes encapsulés aient été décapsulés.

Ainsi, un seul paquet initial déclenche une série d'échanges successifs entre A et B , chaque hôte consommant de la bande passante et des ressources CPU à chaque étape.

Formalisation et facteur d'amplification

Soit :

- m : taille du paquet initial envoyé par l'attaquant (en octets).
- a : taille minimale d'un en-tête IP (20 octets pour IPv4, 40 octets pour IPv6).
- $h = \lfloor m/a \rfloor$: nombre maximal d'en-têtes IP emboîtés dans le paquet.

À chaque décapsulation, un en-tête de taille a est retiré et le paquet restant est réémis. Le premier hôte reçoit initialement un paquet de taille m , puis traite successivement des paquets de taille décroissante. Le volume total de trafic θ_1 reçu et émis par le premier hôte est approximé par :

$$\theta_1 \approx \sum_{k=0}^h (m - a \cdot k) \approx \frac{h \cdot m}{2}$$

Le facteur d'amplification f , défini comme le rapport entre le trafic total traité par un hôte et le trafic initial envoyé par l'attaquant, est donc :

$$f \approx \frac{\theta_1}{m} \approx \frac{h}{2}$$

Il est important de noter que ce facteur d'amplification s'applique à un seul paquet initial. L'attaquant peut multiplier l'impact total de l'attaque en envoyant plusieurs paquets de ce type.

Exemple. Avec une MTU de 1500 octets et IPv4 ($a = 20$), on obtient $h = 75$ et donc un facteur d'amplification $f \approx 37,5$. Le premier hôte traite alors environ 56 250 octets de trafic à partir d'un seul paquet initial de 1500 octets. Avec un paquet de 3000 octets, le facteur d'amplification atteint $f \approx 75$ [1].

Limites théoriques

La taille maximale d'un datagramme IPv4 est de 65 535 octets, ce qui impose une limite supérieure théorique de $h \approx 3276$ en-têtes et donc un facteur d'amplification maximal d'environ 1638. Pour IPv6, cette limite est divisée par deux en raison de la taille plus grande des en-têtes, donnant un facteur d'amplification maximal d'environ 819.

La fragmentation IPv4 peut être exploitée pour approcher cette taille maximale malgré une MTU de chemin plus faible, tandis qu'IPv6 interdit la fragmentation intermédiaire, imposant une limite plus stricte.[1]

Pourquoi l'attaque fonctionne

Cette attaque est rendue possible par plusieurs propriétés fondamentales :

- Absence d'authentification : les hôtes vulnérables décapsulent et traitent tout paquet reçu sans validation de l'expéditeur.
- Inefficacité du TTL : chaque décapsulation révèle un nouvel en-tête IP interne possédant son propre champ TTL, empêchant l'expiration naturelle de la boucle.
- Alternance symétrique des adresses : la construction du paquet garantit que chaque hôte considère le paquet décapsulé comme légitime et destiné à l'autre hôte.

Impact et discussion

Cette attaque permet :

- Un déni de service par épuisement de bande passante et de ressources CPU, le trafic entrant et sortant étant approximativement équivalent.
- Une attaque par déni économique de durabilité (EDoS) dans les environnements cloud où le trafic sortant est facturé.
- Une généralisation à $n > 2$ hôtes, augmentant encore la durée et l'impact global de l'attaque.

Cette attaque est particulièrement pernicieuse car elle transforme des hôtes légitimes en générateurs involontaires de trafic malveillant, sans qu'ils ne reçoivent eux-mêmes un volume de trafic entrant anormalement élevé.

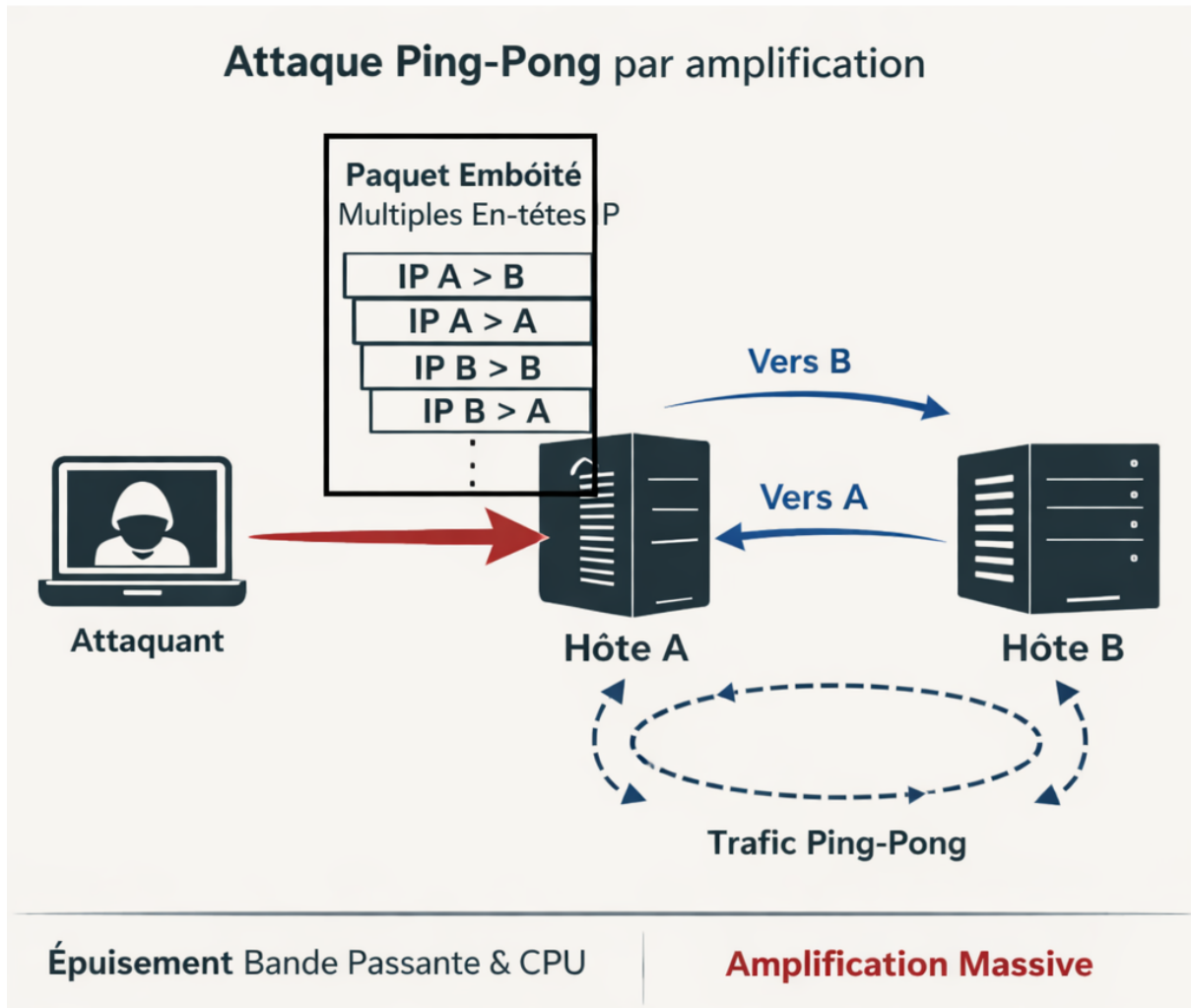


FIGURE 4 – Schéma de l’attaque Ping-Pong par amplification via encapsulation IPIP.

5.4 Attaque Tunnelled Temporal Lensing (TuTL)

L’attaque *Tunnelled Temporal Lensing* (TuTL) est une attaque de déni de service fondée sur la concentration temporelle du trafic réseau plutôt que sur une augmentation du volume brut. Elle exploite des hôtes de tunneling vulnérables afin de créer artificiellement plusieurs chemins réseau de latences différentes vers une victime, puis planifie précisément l’envoi du trafic pour synchroniser l’arrivée des paquets au niveau de la cible.

Modèle d’attaque

Le modèle d’attaque TuTL repose sur les hypothèses suivantes :

- l’attaquant contrôle une machine M capable d’envoyer du trafic tunnelisé ;

- un ensemble d’hôtes intermédiaires vulnérables au tunneling est accessible publiquement ;
- la victime V expose un service répondant aux requêtes entrantes, permettant d’estimer les latences réseau ;
- les latences entre les différents acteurs sont relativement stables à court et moyen terme.

Les hôtes vulnérables ne sont pas compromis au sens classique : ils se comportent comme des relais involontaires, encapsulant et décapsulant les paquets reçus conformément au protocole de tunneling qu’ils supportent.

Phases de l’attaque

L’attaque TuTL se déroule en trois phases principales :

1. Collecte des latences L’attaquant mesure trois types de latences à l’aide de paquets de sondage tunnelisés :

a) **Latence attaquant $\rightarrow$ hôte vulnérable (α) :**

Afin d’estimer la latence entre la machine de l’attaquant M et un hôte vulnérable A , l’attaquant envoie un paquet encapsulé vers A . Après réception, l’hôte vulnérable décapsule le paquet et retransmet le paquet interne vers M . Le chemin suivi par le paquet est donc :

$$M \rightarrow A \rightarrow M$$

L’attaquant mesure alors le temps d’aller-retour (Round-Trip Time, RTT) correspondant à ce trajet et estime la latence unidirectionnelle α de la manière suivante :

$$\alpha \approx \frac{RTT(M \rightarrow A \rightarrow M)}{2}$$

Cette estimation repose sur l’hypothèse que la latence est symétrique entre les deux directions du chemin.

b) **Latence entre deux hôtes vulnérables (β) :**

Pour estimer la latence entre deux hôtes vulnérables A et B , l’attaquant envoie une sonde tunnelisée qui suit le chemin :

$$M \rightarrow A \rightarrow B \rightarrow M$$

La sonde traverse successivement les deux hôtes avant de revenir vers la machine de l’attaquant. L’attaquant mesure alors le RTT correspondant à ce trajet complet. En retranchant les latences déjà estimées entre M et chacun des hôtes (notées α_A et α_B), il peut isoler la latence β entre A et B :

$$\beta = RTT(M \rightarrow A \rightarrow B \rightarrow M) - \alpha_A - \alpha_B$$

Contrairement au cas précédent, cette estimation ne suppose pas nécessairement une symétrie parfaite entre les chemins $A \rightarrow B$ et $B \rightarrow A$. Pour améliorer la précision, les latences individuelles sont donc estimées séparément.

c) **Latence hôte vulnérable → victime (c) :**

Pour estimer la latence c entre un hôte vulnérable, par exemple A et une victime V , l'attaquant procède en plusieurs étapes. Il commence par mesurer la latence v entre sa machine M et la victime V . Cette mesure suppose que V expose un service accessible publiquement (par exemple un service web répondant par un SYN/ACK à un SYN TCP), permettant ainsi de calculer un temps d'aller-retour entre M et V .

Dans un second temps, l'attaquant envoie une sonde encapsulée suivant le chemin :

$$M \rightarrow A \rightarrow V \rightarrow M$$

Le paquet interne contient une adresse source usurpée correspondant à celle de M , de sorte que la réponse générée par la victime soit renvoyée vers l'attaquant. En mesurant le RTT associé à ce trajet complet, l'attaquant obtient une estimation combinée des latences des différents segments.

La latence c entre l'hôte vulnérable et la victime peut alors être estimée en soustrayant la latence α (entre M et A) ainsi que la latence directe v (entre M et V) :

$$c = RTT(M \rightarrow A \rightarrow V \rightarrow M) - \alpha - v$$

Afin de limiter l'impact des variations réseau et des valeurs aberrantes, plusieurs mesures successives (par exemple 1000 RTT) peuvent être réalisées, et la médiane retenue comme estimation finale de la latence.

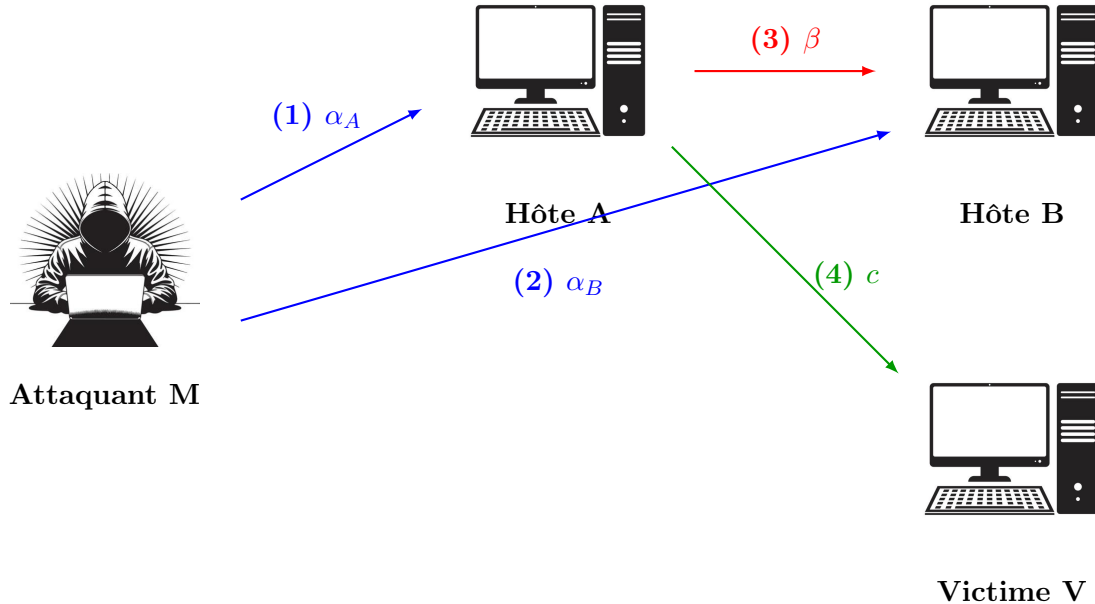


FIGURE 5 – Mesure progressive des latences : (1) et (2) estimation des latences attaquant-hôtes (α), (3) latence inter-hôtes (β), (4) latence vers la victime (c).

2. Exploration des chemins : Un algorithme de parcours en largeur (Breadth-First Search - BFS) est utilisé afin d'identifier systématiquement des chemins vers la victime, en augmentant progressivement leur profondeur d , c'est-à-dire le nombre d'hôtes vulnérables traversés.

L'algorithme procède de la manière suivante :

- Il génère des chemins de profondeur croissante : d'abord $M \rightarrow A \rightarrow V$ (profondeur 1), puis des chemins incluant deux hôtes vulnérables, et ainsi de suite.
- À chaque profondeur, seuls les chemins introduisant une variation de latence suffisante sont conservés. Si plusieurs chemins présentent des latences trop proches, un seul est retenu. Le seuil minimal de différence de latence est fixé à 10 ms.
- Pour chaque chemin sélectionné, l'algorithme comptabilise le volume d'octets que chaque hôte intermédiaire devra traiter. Si un hôte dépasse une limite prédéfinie, il ne peut plus être inclus dans de nouveaux chemins, afin d'éviter tout risque de congestion ou de déni de service involontaire.
- À chaque niveau de profondeur, l'ordre d'exploration des hôtes candidats est aléatoirement mélangé, ce qui permet de répartir la charge de trafic de manière plus homogène.

La contrainte sur le nombre maximal d'octets traités par hôte constitue également une condition d'arrêt implicite : si le nombre d'hôtes encore disponibles devient insuffisant pour construire des chemins d'une profondeur donnée, l'exploration s'interrompt.

La latence totale ℓ d'un chemin de profondeur d est calculée comme la somme :

$$\ell = \alpha + \sum_{i=1}^{d-1} \beta_i + c$$

où α est la latence entre l'attaquant et le premier hôte, β_i la latence entre deux hôtes consécutifs du chemin, et c la latence entre le dernier hôte et la victime.

3. Planification et envoi : Les chemins sont triés par latence croissante ou décroissante ($\ell_1 > \ell_2 > \dots > \ell_N$). L'attaquant calcule les temps de commutation :

$$st_i = \ell_i - \ell_{i+1}$$

Il envoie d'abord des rafales de paquets sur le chemin le plus lent (ℓ_1), attend st_1 millisecondes, puis envoie sur le chemin suivant, et ainsi de suite. Cette séquence garantit que les paquets arrivent simultanément chez la victime.

Formalisation de l'amplification temporelle

L'efficacité de cette concentration temporelle est mesurée à l'aide d'un facteur d'amplification f :

$$f = \frac{b_v}{b_a}$$

où b_a correspond au débit moyen de l'attaquant sur toute la durée d'envoi, et b_v , au débit maximal observé chez la victime dans une fenêtre de 20 ms. Ce facteur capture l'intensité du pic de trafic subi par la victime.[1]

Attaque Tunnelled Temporal Lensing (TuTL)

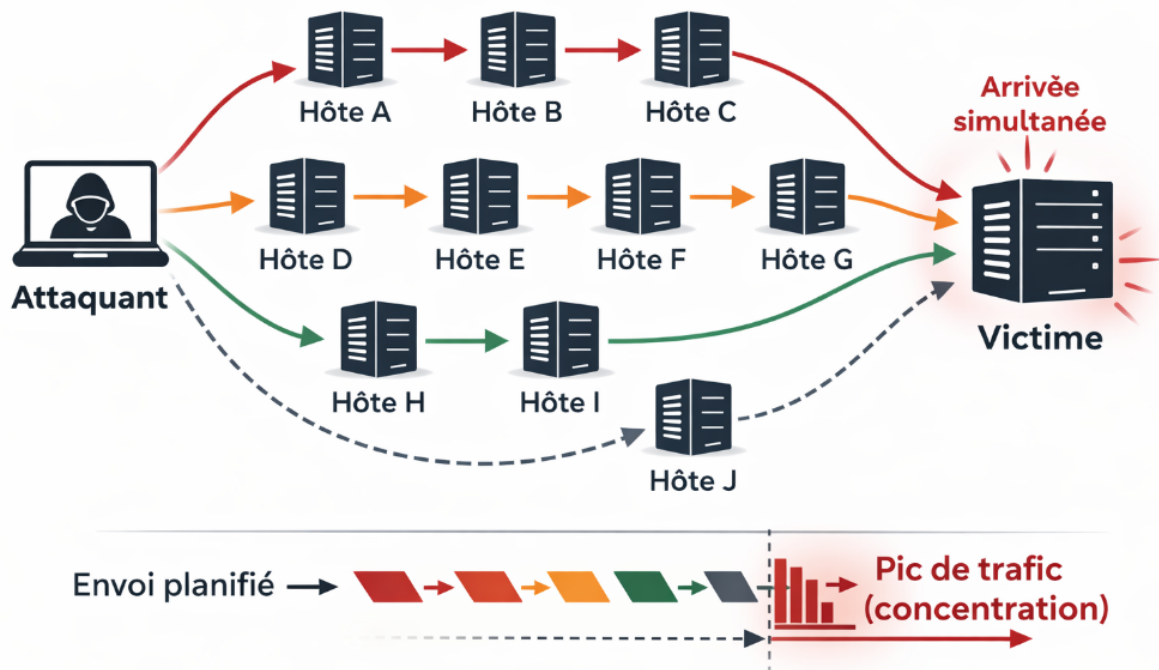


FIGURE 6 – Principe de l’attaque Tunnelled Temporal Lensing (TuTL) : création de chemins de latence différentes et synchronisation temporelle du trafic vers la victime.

Résultats expérimentaux

Les expérimentations rapportées dans l’article montrent que l’attaque TuTL produit des facteurs d’amplification significatifs même avec un nombre réduit d’hôtes vulnérables :

- avec 2 hôtes, un facteur d’amplification moyen de 11 ($\sigma = 2$) ;
- avec 5 hôtes, un facteur d’amplification moyen de 16 ($\sigma = 2,6$).

Ces résultats indiquent que l’augmentation du nombre d’hôtes vulnérables améliore la diversité des chemins disponibles et permet une synchronisation plus fine du trafic, conduisant à un pic de trafic plus élevé chez la victime.[1]

La Figure 7 illustre un exemple d’expérimentation menée avec deux hôtes vulnérables. Elle met en évidence la concentration temporelle du trafic chez la victime, malgré un envoi étalé du côté de l’attaquant.

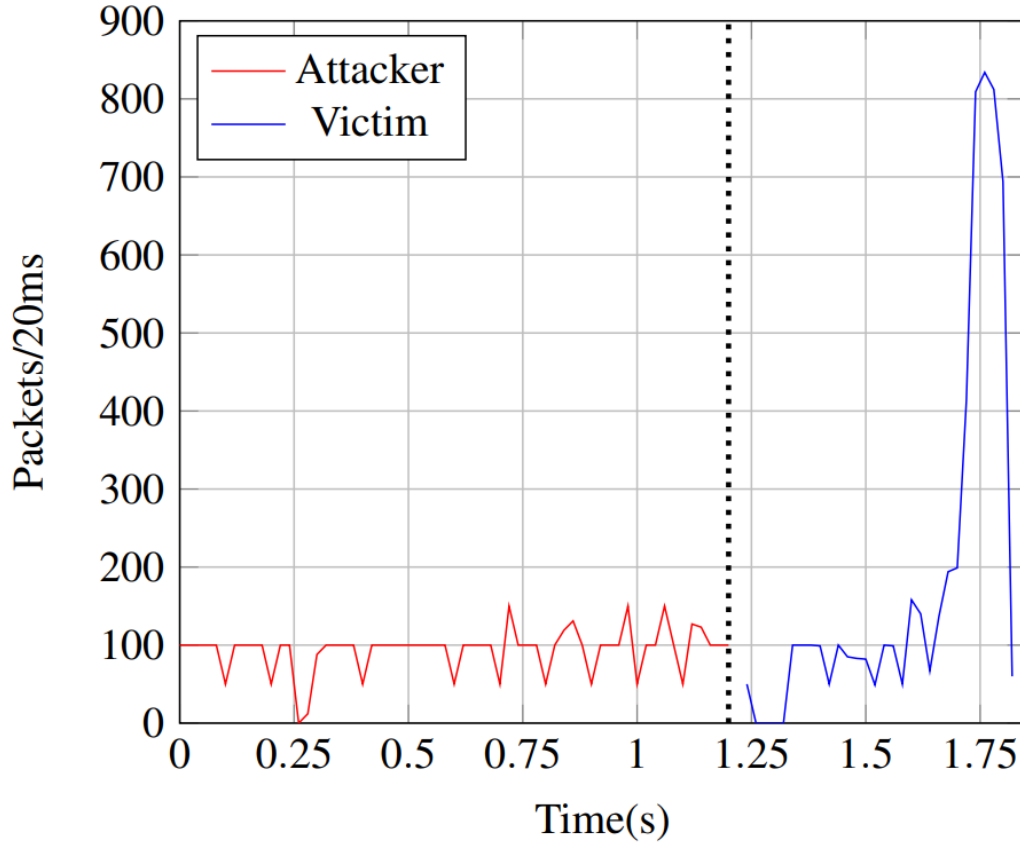


FIGURE 7 – Expérimentation TuTL avec deux hôtes vulnérables (5700 paquets, 16 chemins).

Dans cette expérience, 5700 paquets sont envoyés par l’attaquant en exploitant 16 chemins distincts. L’axe horizontal représente le temps, synchronisé entre la machine de l’attaquant et celle de la victime. À gauche de la ligne pointillée figure le nombre de paquets envoyés par l’attaquant, tandis qu’à droite apparaît le nombre de paquets reçus par la victime, regroupés par fenêtres de 20 ms. On observe que, bien que l’envoi côté attaquant soit relativement étalé dans le temps, les paquets convergent de manière beaucoup plus concentrée chez la victime. Cette concentration illustre directement le phénomène d’amplification temporelle décrit précédemment.

Synthèse

L’attaque TuTL démontre qu’il est possible de provoquer un pic de trafic significatif en exploitant la stabilité relative des latences réseau et la capacité à construire des chemins tunnelisés de longueur contrôlée. En synchronisant précisément l’envoi de paquets sur plusieurs trajectoires de latence différente, l’attaquant transforme un débit moyen modéré en une concentration temporelle intense chez la victime, contournant ainsi les mécanismes de défense fondés sur des seuils moyens. Cette efficacité repose toutefois sur certaines conditions : la disponibilité d’hôtes de tunneling acceptant du trafic sans authentification, la stabilité des délais réseau sur la durée de l’expérience, ainsi que des contraintes techniques telles que la MTU, la bande passante des intermédiaires et la réduction progressive de la taille utile des paquets lors des encapsulations successives.

6 Contre-mesures et bonnes pratiques

Afin de limiter ou prévenir les attaques détaillées précédemment, il est nécessaire de mettre en place des mesures de protection adaptées. Les vulnérabilités liées aux mécanismes de tunnelling peuvent être exploitées de différentes manières, ce qui implique une approche de défense structurée et cohérente.

Les contre-mesures envisageables peuvent être regroupées en deux grandes catégories : celles mises en œuvre directement au niveau des hôtes, et celles appliquées à l'échelle du réseau.

6.1 Défenses au niveau hôte

Les défenses au niveau hôte consistent à appliquer directement les mécanismes de protection sur les systèmes eux-mêmes, sans dépendre exclusivement de l'infrastructure réseau. L'objectif est de limiter l'exposition des interfaces de tunnelling et de réduire la surface d'attaque en contrôlant précisément les paquets acceptés et décapsulés.

Une première mesure consiste à restreindre l'acceptation des paquets de tunnelling aux seules adresses IP sources considérées comme légitimes. Concrètement, un hôte peut maintenir une liste d'adresses autorisées, par exemple basée sur des communications antérieures ou sur une configuration statique définissant les pairs attendus. Tout paquet encapsulé provenant d'une adresse non répertoriée serait alors rejeté avant décapsulation. Une telle approche permet de limiter l'exposition aux scans à grande échelle et d'éviter l'acceptation de paquets provenant de sources arbitraires.

Dans le cas de configurations multipoint telles que mGRE (Multipoint Generic Routing Encapsulation), où l'adresse distante n'est pas explicitement définie, cette problématique devient plus critique. L'absence d'adresse de pair prédéfinie implique que le dispositif peut théoriquement accepter des paquets encapsulés provenant de multiples sources. Il est alors indispensable de mettre en place des règles de filtrage au niveau d'un pare-feu local afin de bloquer les paquets encapsulés dont l'adresse source ne correspond pas à un voisin attendu. Cette vérification doit intervenir avant toute opération de décapsulation afin d'éviter que l'hôte ne serve de relais involontaire. Certains mécanismes comme l'utilisation d'une clé GRE non par défaut peuvent réduire la visibilité d'un hôte lors de scans automatisés. Toutefois, ce champ de clé ne constitue pas un mécanisme d'authentification robuste. En l'absence de chiffrement, la clé peut être observée dans des paquets en clair ou retrouvée par brute force. Elle ne fournit donc qu'une protection limitée face à un adversaire motivé.

Il convient également de souligner que le simple filtrage par adresse IP ne protège pas contre un attaquant capable d'usurper une adresse source autorisée. En effet, ces protocoles 2 de tunnelling n'intègrent pas nativement de mécanisme d'authentification ou de vérification d'intégrité. Un attaquant en mesure d'usurper l'adresse IP source peut ainsi contourner une politique de confiance basée uniquement sur l'adressage.

Pour obtenir un niveau de sécurité plus élevé, il est recommandé d'associer les protocoles de tunnelling à des mécanismes cryptographiques assurant à la fois l'authentification et le chiffrement. Une solution largement standardisée est l'utilisation d'IPsec (internet Protocol Security), défini notamment dans l'architecture décrite par la RFC 4301 [10]. IPsec est un ensemble de protocoles fournissant des services de confidentialité, d'intégrité

et d'authentification au niveau IP. Il repose principalement sur deux en-têtes : l'Authentication Header (AH) pour l'intégrité et l'authentification, et l'Encapsulating Security Payload (ESP) pour le chiffrement et/ou l'intégrité.

Dans le contexte du tunnelling, IPsec peut être utilisé de deux manières principales. En mode transport, IPsec protège la charge utile du paquet IP sans encapsuler un nouvel en-tête IP externe. En mode tunnel, en revanche, l'intégralité du paquet IP interne (en-tête et données) est encapsulée et protégée par un nouvel en-tête IP externe. Ce second mode est particulièrement adapté à la sécurisation de tunnels tels que IPIP ou GRE, car il permet de chiffrer et d'authentifier le paquet interne avant sa transmission.

Concrètement, lorsqu'IPsec est utilisé pour protéger un tunnel, le paquet encapsulé (par exemple un paquet GRE contenant un paquet IP interne) est lui-même inséré dans une association de sécurité IPsec (Security Association). Cette association définit les algorithmes cryptographiques utilisés, les clés partagées et les paramètres de sécurité négociés via le protocole IKE (internet Key Exchange). Le système ne doit alors accepter que les paquets encapsulés protégés par une association IPsec valide. Tout paquet de tunnelling reçu en clair, sans protection IPsec, doit être rejeté[10].

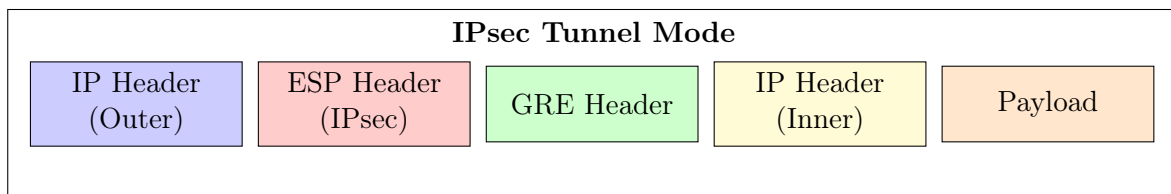


FIGURE 8 – Encapsulation GRE protégée par IPsec en mode tunnel

L'intégration d'IPsec permet ainsi de transformer un mécanisme de tunnelling initialement dépourvu d'authentification en un canal sécurisé assurant confidentialité, intégrité et vérification de l'origine des paquets. Cette approche constitue la solution la plus robuste pour prévenir l'exploitation des vulnérabilités liées aux tunnels ouverts.

6.2 Défenses au niveau réseau

Les défenses réseau visent à prévenir l'exploitation des vulnérabilités de tunnelling en amont, avant que les paquets malveillants n'atteignent les hôtes cibles. Ces mécanismes complètent les protections locales et réduisent la surface d'attaque globale.

Filtrage des adresses source

Le filtrage des adresses source, standardisé dans la RFC 2827 (BCP38), est la première ligne de défense contre l'usurpation d'adresse IP. Il consiste à vérifier que les paquets entrants et sortants possèdent des adresses sources légitimes, c'est-à-dire appartenant aux plages IP allouées au réseau local. Cette mesure, simple mais efficace, limite drastiquement la capacité d'un attaquant à masquer son identité ou à exploiter des hôtes vulnérables pour relayer du trafic falsifié.

Deux types de filtrage sont appliqués :

- **Egress filtering** : Les paquets sortants dont l'adresse source n'appartient pas au réseau local sont rejetés.

- **Ingress filtering** : Les paquets entrants prétendant provenir d'adresses internes sont bloqués.

Cette défense neutralise : les attaques par usurpation d'adresse source (ex. : scans par usurpation complète), les abus de signalement, en empêchant l'envoi de trafic falsifié depuis le réseau et une partie des attaques par amplification (ex. : Ping-Pong), en rendant l'attaquant traçable.

Inspection approfondie des paquets (DPI)

L'inspection approfondie des paquets (Deep Packet Inspection, DPI) constitue un moyen efficace pour identifier des comportements suspects dans le trafic tunnelisé. Elle permet notamment de détecter des encapsulations récursives comportant plus de deux ou trois en-têtes IP emboîtés, ce qui peut révéler des mécanismes d'abus tels que l'attaque Ping-Pong décrite précédemment dans 5.3 ; le seuil retenu doit toutefois être ajusté en fonction de l'architecture légitime du réseau. L'analyse des champs TTL ou Hop Limit des en-têtes internes peut également mettre en évidence des valeurs anormalement faibles (0 ou 1), caractéristiques de techniques de scan exploitant l'expiration du TTL. Enfin, la présence d'adresses IPv6 malformées, notamment des motifs du type `::IPv4_ADDRESS`, peut signaler des tentatives d'exploitation de boucles de routage, comme décrit dans la section 5.1. Des outils tels que Suricata ou Zeek peuvent être configurés afin d'automatiser la détection de ces anomalies à travers des règles spécifiques.

Néanmoins, le recours à la DPI présente certaines limites : elle peut introduire un surcoût en termes de performances, notamment sur des liens à fort débit, et nécessite une mise à jour régulière des signatures et des règles afin de rester efficace face à l'évolution des techniques d'attaque.

"

Exemple de paquet IPIP bloqué par DPI

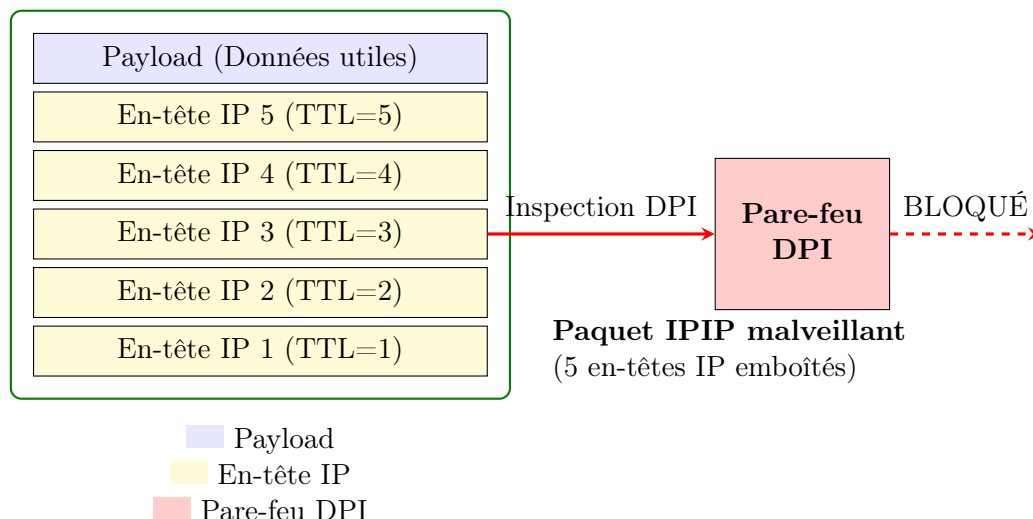


FIGURE 9 – Paquet IPIP avec 5 en-têtes emboîtés

Blocage des tunnels non chiffrés

Seuls les tunnels protégés par des mécanismes de sécurité robustes (ex. : IPsec/ESP) doivent être autorisés. Tout trafic de tunnelling en clair (GRE, IPIP, 6in4, etc) est bloqué

par défaut, sauf exception justifiée.

Exemple de politique réseau

- **Autoriser** : Protocole ESP (IPsec, numéro 50).
- **Bloquer** : Protocoles GRE (47), IPIP (4), IPv6-in-IPv4 (41).

TABLE 2 – Politique de filtrage des protocoles de tunnelling

Protocole	Numéro IP	Politique par défaut	Justification
ESP (IPsec)	50	Autoriser	Tunnel sécurisé et authentifié.
GRE	47	Bloquer	Non chiffré, vulnérable aux abus.
IPIP	4	Bloquer	Risque d'usurpation et d'amplification.
6in4	41	Bloquer	Transition non sécurisée.

Limitation de débit adaptée aux attaques temporelles

Les mécanismes traditionnels de limitation de débit, fondés sur des seuils moyens (par exemple un nombre maximal de paquets par seconde), se révèlent insuffisants face à des attaques comme **TuTL**, qui reposent sur la concentration du trafic dans des fenêtres temporelles extrêmement courtes, typiquement de l'ordre de 20 ms. Dans ce contexte, un attaquant peut générer des pics très intenses tout en restant sous les limites moyennes configurées sur une seconde complète. Pour contrer ce type de stratégie, il est nécessaire d'adopter une approche plus fine consistant à mesurer et contrôler le débit sur des intervalles courts, notamment des fenêtres glissantes de 20 ms, afin de détecter et bloquer les concentrations anormales avant qu'elles n'atteignent la cible. Une telle adaptation permet de neutraliser efficacement l'amplification temporelle exploitée par TuTL, mais elle suppose l'utilisation d'équipements capables de traiter des métriques à granularité élevée, tels que des pare-feu ou des routeurs modernes à haut débit [1, 13].

7 Conclusion

Au terme de cette étude, il apparaît clairement que les protocoles de tunnelling, bien qu'essentiels au fonctionnement et à l'évolution d'internet, constituent encore aujourd'hui une surface d'attaque largement sous-estimée. L'analyse des principaux mécanismes comme IPIP, GRE, 6in4, 4in6 et GUE, montre que leur conception historique, fondée sur un modèle de confiance implicite, les rend particulièrement vulnérables lorsqu'ils sont déployés sans mécanismes de protections supplémentaires. Le manque d'authentification et de chiffrement par défaut engendre un grand écart entre les utilisations prévues et celles réellement constatées sur internet.

Les résultats expérimentaux issus de l'article [1] confirment l'ampleur du phénomène : plus de 4 millions d'hôtes IPv4 et IPv6 acceptent du trafic encapsulé non authentifié depuis des sources arbitraires. Cette large exposition prouve que le problème n'est pas seulement théorique, mais aussi systémique. Elle affecte diverses infrastructures, dispersées dans de multiples systèmes autonomes et régions du monde, et remet en cause l'efficacité des méthodes traditionnelles de filtrage des adresses source lorsque celles-ci peuvent être contournées indirectement par le biais d'hôtes de tunneling accessibles.

Au-delà du simple abus comme proxy unidirectionnel, notre étude met en évidence des impacts sécuritaires concrets et parfois graves : des hôtes mal configurés permettant l'usurpation d'adresses IP, le contournement de politiques de filtrage, la génération de faux signalements administratifs, et nouvelles formes d'attaques par déni de service. Les attaques de type Ping-Pong, Tunnelled Temporal Lensing (TuTL) illustrent comment des propriétés protocolaires légitimes peuvent être détournées pour produire des effets d'amplification des attaques, que ce soit en termes de volume, de concentration temporelle ou de coûts financiers pour la victime. Ainsi, le tunnelling ne se résume plus à un simple transit de paquets : il peut se transformer en un véritable amplificateur pour un assaillant, convertissant des dispositifs prévus pour la connectivité en vecteurs de vulnérabilité.

Face à ces constats, les contre-mesures présentées montrent qu'il ne s'agit pas d'un problème insoluble, mais d'un problème de configuration, de gouvernance et de bonnes pratiques. La restriction explicite des adresses autorisées, l'activation systématique du filtrage source, l'usage de mécanismes de sécurisation tels que IPsec, ainsi qu'une meilleure visibilité sur les interfaces de tunnelling exposées, constituent des mesures concrètes et immédiatement applicables. Toutefois, leur efficacité dépendra d'une prise de conscience plus large des opérateurs et administrateurs réseau.

En fin, ce travail souligne que les mécanismes hérités du passé continuent d'influencer profondément la sécurité de l'internet d'aujourd'hui. Tant que des protocoles conçus dans un contexte de confiance implicite resteront activés par défaut sans protection adéquate, ils offriront des opportunités d'abus à grande échelle. Il est donc essentiel d'intégrer la sécurité dès la configuration des tunnels et de considérer ces interfaces non comme de simples outils techniques, mais comme des points d'exposition critiques au sein des infrastructures modernes.

8 Bibliographie

Références

- [1] A. Beitis et M. Vanhoef, *Haunted by Legacy : Discovering and Exploiting Vulnerable Tunnelling Hosts*, <https://papers.mathyvanhoef.com/usenix2025-tunnels.pdf>
- [2] C. Perkins, *IP Encapsulation within IP*, <https://datatracker.ietf.org/doc/rfc2003/>
- [3] Yutube, *Tunnel IP*, https://youtu.be/MX-4i9wkRq4?si=_HKyvaMNQHmEYcu3
- [4] D. Farinacci, *Generic Routing Encapsulation (GRE)*, <https://datatracker.ietf.org/doc/rfc2784/>
- [5] Yutube, *Tunnel IPv6 to IPv4*, https://www.youtube.com/watch?v=nGRs3JHkt_g
- [6] Yutube, *6TO4 Tunnel*, https://www.youtube.com/watch?v=0BxwCE2_8EQ
- [7] T. Herbert, *UDP Encapsulation in Linux*, <https://people.netfilter.org/pablo/netdev0.1/papers/UDP-Encapsulation-in-Linux.pdf>
- [8] YouTube, Netdev 0.1, *UDP Encapsulation in Linux*, https://youtu.be/hKTD9W2C5s8?si=f0vB0_1x9Qv7p21b
- [9] CERT Coordination Center, *Vulnerability Note VU636397 : IP-in-IP tunneling implementation flaw*, <https://kb.cert.org/vuls/id/636397>
- [10] S. Frankel & S. Krishnan, *IP Security (IPsec) and internet Key Exchange (IKE) Document Roadmap, RFC 6071*, <https://datatracker.ietf.org/doc/html/rfc6071>
- [11] John Kristoff, Mohammad Ghasemisharif, Chris Kanich, and Jason Polakis <https://dataplane.org/jtk/publications/kgkp-pam-21.pdf>
- [12] Yannay Livneh <https://mcfp.felk.cvut.cz/publicDatasets/pocorgtfo/contents/articles/21-03.pdf>
- [13] Marc Kühner, Thomas Hupperich, Christian Rossow, and Thorsten Holz, Ruhr-University Bochum <https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-kuhrer.pdf>